

I. INTRODUCTION

Dans le cadre de ce projet, nous avons pour objectif de concevoir un algorithme de recherche de cible à l'aide d'un drone, dans un environnement rectangulaire défini. Le drone évolue dans une salle dont les dimensions sont connues, mais la position de la cible lui est inconnue. Celle-ci n'est détectable que lorsque le drone s'en approche à moins de 50 cm.

Ce projet nous amène à développer une première version de l'algorithme en pseudo-code, puis à l'implémenter en Python. Une version généralisée de l'algorithme est également demandée pour fonctionner dans toute salle de dimensions quelconques.

Ceci dit, nous allons procéder à l'exécution des différentes tâches :

II. Pseudo-code qui explore la salle tant que la cible n'a pas été détectée

La première étape de ce projet consiste à concevoir un algorithme, sous forme de pseudo-code, permettant à un drone d'explorer de manière autonome une salle rectangulaire de dimensions $L_x = 500$, $L_y = 500$ et $H = 250$. L'objectif est de parcourir entièrement la zone afin de détecter une cible inconnue, que le drone ne peut repérer qu'à moins de 50 cm. Cette étape est cruciale car elle pose les bases logiques de l'algorithme de recherche, en définissant une stratégie de déplacement systématique et efficace dans l'espace donné.

Algo_drone

Variables : marges = 52, pas = 50, long_utile, large_utile, distance, nb_ligne : Réels

Début

Large_utile $\leftarrow 500 - 2 \times \text{marge}$

Long_utile $\leftarrow 1000 - 2 \times \text{marge}$

Nb_ligne $\leftarrow \text{large_utile} / \text{pas}$

Locate (x=marge, y=marge, heading=90)

Takeoff ()

GoUp (100)

Pour i allant de 0 à nb_ligne - 1 faire

distance $\leftarrow 0$

Tant que distance < long_utile faire

forward(pas)

distance $\leftarrow \text{distance} + \text{pas}$

Si isTargetDetected() alors

```

        Afficher ("la cible a été détectée")
        land ()
    Fsi
FTant que
    Si i < nb_ligne - 1 alors
        Si i / 2 = 0 alors
            rotateRight(90)
            forward(pas)
            rotateRight(90)
        Sinon
            rotateLeft(90)
            forward(pas)
            rotateLeft(90)
        FSi
    FSi
FPour
    Afficher ("la cible n'a pas été trouvée")
    Land ()
Fin Algo_drone

```

III. Implémentation de l'algorithme sous Pycharm (Python)

Après avoir défini notre algorithme en pseudo-code, l'étape suivante consiste à le transcrire en langage PYTHON afin de le rendre exécutable. L'objectif est de simuler le comportement d'un drone explorant une salle de dimensions fixes ($L_x = 500$, $L_y = 500$, $H = 250$) pour détecter une cible à une distance inférieure ou égale à 50 cm. Cette implémentation permettra de tester la logique de notre algorithme dans un environnement de développement réel et d'observer son efficacité dans la recherche de la cible.

1. Code source :

Pour cela, nous allons vous présenter l'ensemble du programme avant de procéder à la décortication du contenu :

```
main.py test06.py teste2.py x
1 > import ...
3
4 def monprogramme(): 1 usage
5
6     try:
7         # PARAMÈTRES
8         marge = 50      # Pour que le drone ne touche pas le mur
9         pas = 50        # les pas de déplacement de 50 cm
10
11         large_utile = 500 - 2*marge    # X utile
12         long_utile = 1000 - 2*marge    # Y utile
13
14         | #INITIALISATION
15         locate(x=marge, y=marge, heading=90)
16         takeOff()
17         goUp(100)
18
19         nb_ligne = large_utile // pas
20
21         for ligne in range(nb_ligne):
22
23             # Avance sur la longueur utile
24             distance_parcourue = 0
25             while distance_parcourue < long_utile:
26                 forward(pas)
27                 distance_parcourue += pas
28
29             # Détection de la cible
30             if isTargetDetected():
31                 print("la Cible a été détectée")
32                 land()
33                 return
34
35
36         # Fin de ligne : demi-tour + décalage latéral
37         if ligne < nb_ligne - 1:
38             if ligne % 2 == 0:
39                 rotateRight(90)
40                 forward(pas)
41                 rotateRight(90)
42             else:
43                 rotateLeft(90)
44                 forward(pas)
45                 rotateLeft(90)
46
47         print("la Cible n'a pas été trouvée")
48         land()
49
50     except Exception as err:
51         print(err)
52         display()
53         traceback.print_exc()
54
55 if __name__ == '__main__':
56     print("**** TEST n°6 : mouvements de base, en passant par une fonction dans VIEWER_TKMPL.")
57     createRoom(description: "(0 0, 500 0, 500 1000, 0 1000, 0 0)", height: 300)
58     createTargetIn(x1: 200, y1: 200, z1: 150, x2: 300, y2: 300, z2: 200)
59     createDrone(DRONE_VIRTUAL, VIEWER_TKMPL, progfunc=monprogramme)
```

Nous remarquons qu'il n'y a pas une grande différence entre le pseudo code et la version implémentée, puisqu'il s'agit d'une simple traduction ; néanmoins nous allons éclaircir les différentes étapes :

```

main.py  test05.py  test02.py
> import ...

def monprogramme(): 1 usage

    try:
        # PARAMÈTRES
        marge = 50    # Pour que le drone ne touche pas le mur
        pas = 50      # les pas de déplacement de 50 cm

        large_utile = 500 - 2*marge    # X utile
        long_utile = 1000 - 2*marge    # Y utile

```

Après avoir définie la fonction qui va contenir mon programme principal, nous avons initialiser quelques variables

➤ **Marge** : définit une distance de sécurité par rapport aux murs de la salle. Elle permet d'éviter les

collisions avec le mur.

- **Pas** : correspond à la distance parcourue par le drone à chaque déplacement. Cette distance peut-être modifier, tout en tenant compte des dimensions de la salle.
- **Large_utile** : Cette variable définit la largeur réellement exploitable de la salle sur l'axe X. On a retiré la marge d'une part et d'autre de la largeur total de la salle.
- **Long_utile** : même principe que la largeur utile

```

3
4      #INITIALISATION
5      locate(x=marge, y=marge, heading=90)
6      takeOff()
7      goUp(100)
8
9      nb_ligne = large_utile // pas

```

Cette instruction positionne le drone au point de départ de la mission.

X = marge : le drone est placé à une distance marge du mur

gauche.

Y = marge :le drone est placé à une distance marge du mur inférieur.

Heading = 90 : le drone est orienté dans la direction de l'axe Y positif (vers l'avant de la salle)

- **locate** est une fonction qui permet de positionner le drone à une position initiale,
- **takeOff** qui lui permet de décoller
- **goUp(100)** de monter de 100cm

```

nb_ligne = large_utile // pas

for ligne in range(nb_ligne):

    # Avance sur la longueur utile
    distance_parcourue = 0
    while distance_parcourue < long_utile:
        forward(pas)
        distance_parcourue += pas

    # Détection de la cible
    if isTargetDetected():
        print("la Cible a été détectée")
        land()
        return

    # Fin de ligne : demi-tour + décalage latéral
    if ligne < nb_ligne - 1:
        if ligne % 2 == 0:
            rotateRight(90)
            forward(pas)
            rotateRight(90)
        else:
            rotateLeft(90)
            forward(pas)
            rotateLeft(90)

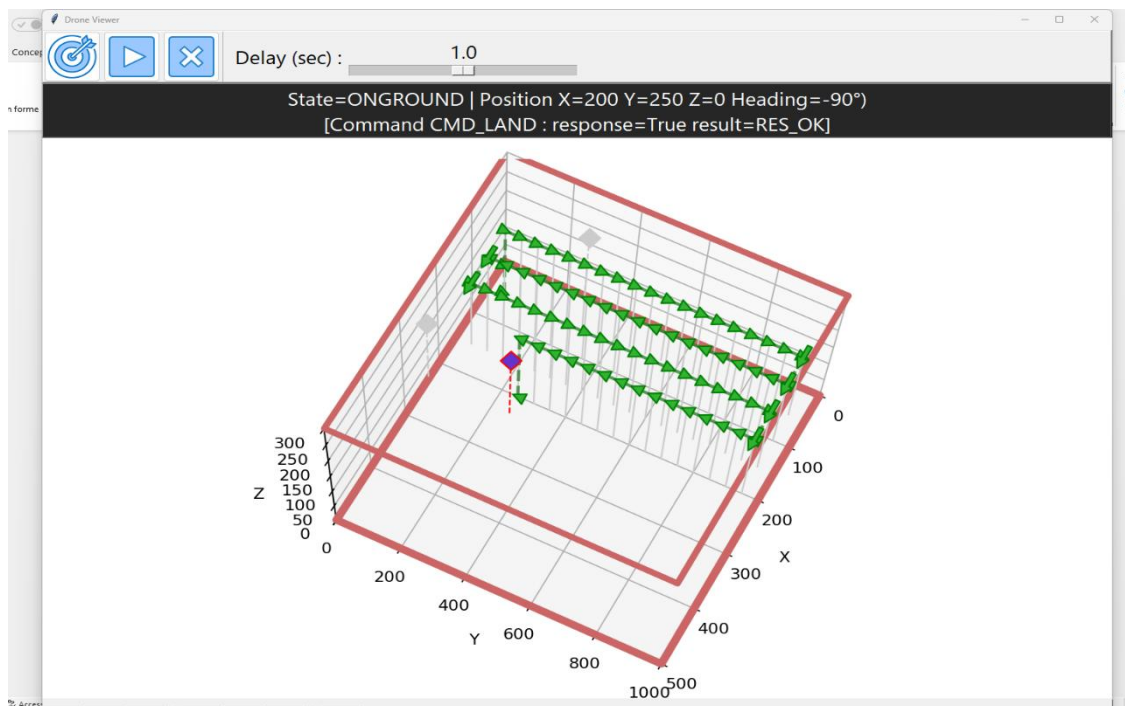
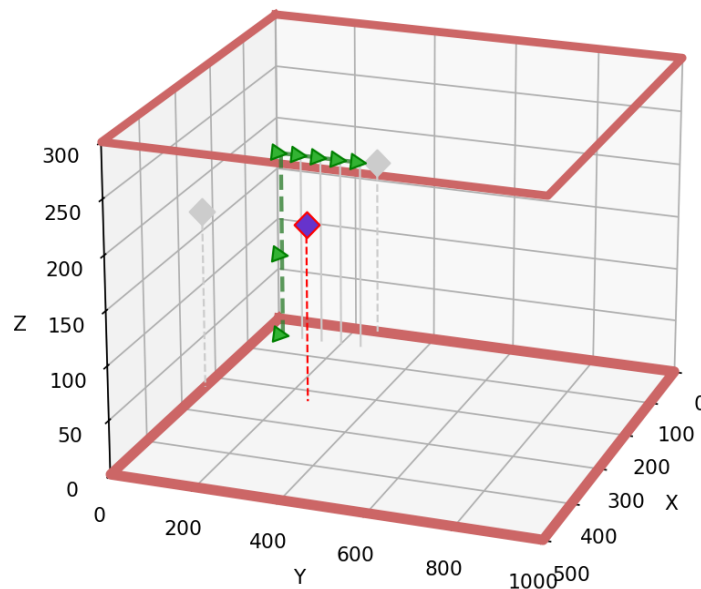
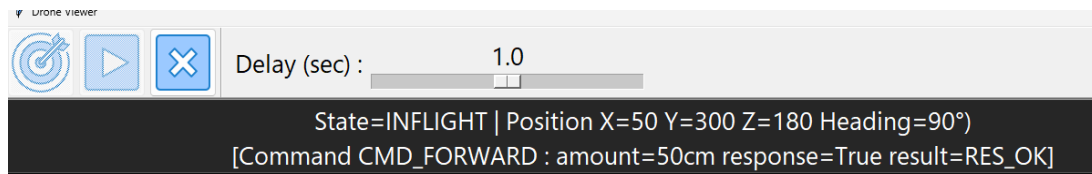
print("la Cible n'a pas été trouvée")
land()

```

C'est la partie la plus importante du programme :

- **for ligne in range(nb_ligne):** Cette boucle correspond au parcours ligne par ligne de la salle. À chaque itération, le drone explore une bande parallèle à la précédente.
- **while distance_parcourue < long_utile:** Cette boucle contrôle l'avancement du drone le long d'une bande. La condition garantit que le drone ne dépasse pas la zone autorisée. **distance_parcourue** mesure la distance déjà parcourue sur la longueur utile.
- **forward(pas) :** le drone avance de pas centimètres, puis la distance parcourue est mise à jour.
- **if isTargetDetected():** cette condition vérifie après chaque déplacement si la cible est détectée.
- **if ligne < nb_ligne - 1:** cette condition empêche le drone d'effectuer un décalage inutile après la dernière bande.
- **if ligne % 2 == 0 :** Cette condition alterne le sens de déplacement d'une bande à l'autre.
 - Lignes paires : déplacement dans un sens,
 - Lignes impaires : déplacement inverse.
- **rotateRight(90), forward(pas), rotateRight(90) :** ce sont des commandes permettant de se décaler d'une bande à une autre, puis de repartir dans la direction opposée.

2. Exécution sur l'interface



Nous remarquons très bien que lorsque le drone a détecté la cible, il s'est posé comme nous l'avons ordonné dans le programme principal.

IV. Pseudo-code qui explore une salle quelconque tant que la cible n'a pas été détectée

La troisième étape de ce projet consiste à concevoir un algorithme, sous forme de pseudo-code, permettant à un drone d'explorer de manière autonome n'importe quelle salle rectangulaire. L'objectif est de parcourir entièrement la zone afin de détecter une cible inconnue, que le drone ne peut repérer qu'à moins de 50 cm. Cette étape est cruciale car elle pose les bases logiques de l'algorithme de recherche, en définissant une stratégie de déplacement systématique et efficace dans l'espace donné.

Algo_drone

Variables : marge = 50, pas = 50, long_utile, large_utile, distance, nb_ligne, nb_couches, large_salle, long_salle, hauteur_salle, hauteur_utile, distance_y : Réels

Début

Afficher ("création de la salle")

Afficher ("donner la longueur de la salle")

Lire (long_salle)

Afficher ("donner la largeur de la salle")

Lire (large_salle)

Afficher ("donner la longueur de la salle")

Lire (hauteur_salle)

```
room_str = f"(0 0, { large_salle } 0, { large_salle } { long_salle }, 0    { long_salle }, 0 0)"  
createRoom(room_str, HAUTEUR_SALLE)
```

Afficher ("création de la cible")

Afficher ("donner la valeur de Xmin")

Lire (x1)

Afficher ("donner la valeur de Xmax")

Lire (x2)

Afficher ("donner la valeur de Ymin")

Lire (y1)

Afficher ("donner la valeur de Ymax")

Lire (y2)

Afficher (“donner la valeur de Zmin”)

Lire (z1)

Afficher (“donner la valeur de Zmax”)

createTargetIn (x1, y1, z1, x2, y2, z2)

Lire (z2)

Large_utile $\leftarrow 500 - 2 \times \text{marge}$

Long_utile $\leftarrow 1000 - 2 \times \text{marge}$

nb_ligne $\leftarrow \text{large_utile} / \text{pas}$

nb_couche $\leftarrow \text{hauteur_utile} / \text{pas}$

Locate (x=marge, y=marge, heading=90)

takeOff()

goUp(marge)

Pour couche allant de 0 à nb_couche - 1, faire

 Pour ligne allant de 0 à nb_ligne - 1, faire

 distance_y $\leftarrow 0$

 Tant que distance_y < long_utile faire

 forward(pas)

 distance_y += pas

 SI isTargetDetected() alors

 Afficher ("la Cible a été détectée")

 Land ()

 Fsi

 FTant que

 SI ligne < nb_lignes - 1 alors

 Si ligne / 2 = 0 alors

 rotateRight(90)

 forward(pas)

 rotateRight(90)

N'importe quelle salle.py × test.py

```
1 > import ...
3
4 def monprogramme(): 1 usage
5 | try:
6 |     # ----- PARAMÈTRES DE SÉCURITÉ -----
7 |     marge = 50      # distance au mur (cm)
8 |     pas = 50        # pas de déplacement (cm)
9 |
10 |    # ----- DIMENSIONS UTILES -----
11 |    large_utile = LARGE_SALLE - 2 * marge
12 |    long_utile = LONG_SALLE - 2 * marge
13 |    hauteur_utile = HAUTEUR_SALLE - 2 * marge
14 |
15 |    # ----- CALCUL DU MAILLAGE -----
16 |    nb_lignes = int(large_utile // pas)
17 |    nb_couches = int(hauteur_utile // pas)
18 |
19 |    # ----- INITIALISATION -----
20 |    locate(x=marge, y=marge, heading=90)
21 |    takeOff()
22 |    goUp(marge)
23 |
24 |    # ----- PARCOURS -----
25 |    for couche in range(nb_couches):
26 |        for ligne in range(nb_lignes):
27 |            distance_y = 0
28 |            while distance_y < long_utile:
29 |                forward(pas)
30 |                distance_y += pas
31 |
32 |            # Détection de la cible
33 |            if isTargetDetected():
34 |                print("la Cible a été détectée")
35 |                land()
36 |                return
37 |
38 |            # Demi-tour + décalage latéral
39 |            if ligne < nb_lignes - 1:
40 |                if ligne % 2 == 0:
41 |                    rotateRight(90)
42 |                    forward(pas)
43 |                    rotateRight(90)
```

```
N'importe quelle salle.py × test.py

def monprogramme(): 1 usage
    forward(pas)
    rotateRight(90)
    else:
        rotateLeft(90)
        forward(pas)
        rotateLeft(90)

    # Passage à la couche supérieure
    if couche < nb_couches - 1:
        goUp(pas)
        rotateRight(180) # repositionnement pour nouvelle couche
    print("la Cible n'a pas été trouvée")
    land()

except Exception as err:
    print(err)
    display()
    traceback.print_exc()

# ===== PROGRAMME PRINCIPAL =====
if __name__ == '__main__':

    print("=== Création de la salle ===")
    LARGE_SALLE = int(input("Donner la largeur de la salle (en cm) : "))
    LONG_SALLE = int(input("donner la longueur de la salle (en cm) : "))
    HAUTEUR_SALLE =int(input("donner la hauteur de la salle (en cm) : "))

    room_str = f"(0 0, {LARGE_SALLE} 0, {LARGE_SALLE} {LONG_SALLE}, 0 {LONG_SALLE}, 0 0)"
    createRoom(room_str, HAUTEUR_SALLE)

    print("\n=== Création de la cible ===")
    x1 = int(input("Entrer la valeur minimal de la position de la cible sur l'axe X : "))
    x2 = int(input("Entrer la valeur maximal de la position de la cible sur l'axe X : "))
    y1 = int(input("Entrer la valeur minimal de la position de la cible sur l'axe Y : "))
    y2 = int(input("Entrer la valeur maximal de la position de la cible sur l'axe Y : "))
    z1 = int(input("Entrer la valeur minimal de la hauteur de la cible : "))
    z2 = int(input("Entrer la valeur maximale de la hauteur de la cible :"))

    createTargetIn(x1, y1, z1, x2, y2, z2)
    createDrone(DRONE_VIRTUAL, VIEWER_TKMPL, progfunc=monprogramme)
```

Pour la bonne compréhension, nous allons élucider les différentes étapes :

```
def monprogramme(): 1 usage
    try:
        # ----- PARAMÈTRES DE SÉCURITÉ -----
        marge = 50      # distance au mur (cm)
        pas = 50        # pas de déplacement (cm)

        # ----- DIMENSIONS UTILES -----
        large_utile = LARGE_SALLE - 2 * marge
        long_utile = LONG_SALLE - 2 * marge
        hauteur_utile = HAUTEUR_SALLE - 2 * marge

        # ----- CALCUL DU MAILLAGE -----|
        nb_lignes = int(large_utile // pas)
        nb_couches = int(hauteur_utile // pas)
```

Après avoir définie la fonction qui va contenir mon programme principal, nous avons initialiser quelques variables

- **Marge** : définit une distance de sécurité par rapport aux murs de la salle. Elle permet d'éviter les collisions avec le mur.
- **Pas** : correspond à la distance parcourue par le drone à chaque déplacement. Cette

distance peut-être modifier, tout en tenant compte des dimensions de la salle.

- **Large_utile** : Cette variable définit la largeur réellement exploitable de la salle sur l'axe X. On a retiré la marge d'une part et d'autre de la largeur total de la salle.
- **Long_utile** : même principe que la largeur utile
- **Hateur_utile** : même principe que la largeur utile
- **Nb_lignes et nb_couche** qui définissent respectivement le nombres de lignes à parcourir et le nombres de couches de hauteurs à parcourir

```
# ----- INITIALISATION -----
locate(x=marge, y=marge, heading=90)
takeOff()
goUp(marge)
```

Cette instruction positionne le drone au point de départ de la mission.

X = **marge** : le drone est placé à une distance marge du mur gauche.

Y = **marge** : le drone est placé à une distance marge du mur inférieur.

Heading = 90 : le drone est orienté dans la direction de l'axe Y positif (vers l'avant de la salle)

- **locate** est une fonction qui permet de positionner le drone à une position initiale,
- **takeOff** qui lui permet de décoller
- **goUp(marge)** de monter d'une marge

```
# -----PARCOURS-----
for couche in range(nb_couches):
    for ligne in range(nb_lignes):
        distance_y = 0
        while distance_y < long_utile:
            forward(pas)
            distance_y += pas
```

C'est l'une des parties les plus importantes du programme, puisque c'est ces boucles qui parcourent l'ensemble des lignes et des couches pour trouver la cible

➤ `for couche in range(nb_couche):`

Cette boucle correspond au parcours couche par couche de la salle. À chaque itération, le drone explore une bande parallèle à la précédente

- `for ligne in range(nb_ligne):` Cette boucle correspond au parcours ligne par ligne de la salle. À chaque itération, le drone explore une bande parallèle à la précédente
- `while distance_parcourue < long_utile:` Cette boucle contrôle l'avancement du drone le long d'une bande. La condition garantit que le drone ne dépasse pas la zone autorisée. `distance_y` mesure la distance déjà parcourue sur la longueur utile.

```
# Détection de la cible
if isTargetDetected():
    print("la Cible a été détectée")
    land()
    return
```

C'est une partie du programme qui dit au drone d'atterrir(`land()`) s'il trouve la cible (`isTargetDetected()`)

```
# Demi-tour + décalage latéral
if ligne < nb_lignes - 1:
    if ligne % 2 == 0:
        rotateRight(90)
        forward(pas)
        rotateRight(90)
    else:
        rotateLeft(90)
        forward(pas)
        rotateLeft(90)
```

➤ `if ligne < nb_lignes - 1:` cette condition empêche le drone d'effectuer un décalage inutile après la dernière bande.

➤ `if ligne % 2 == 0 :` Cette condition alterne le sens de déplacement d'une bande à l'autre.

- Lignes paires : déplacement dans un sens,
- Lignes impaires : déplacement inverse.

➤ `rotateRight(90)`, `forward(pas)`, `rotateRight(90)` : ce sont des commandes permettant de se décaler d'une bande à une autre, puis de repartir dans la direction opposée.

```
# Passage à la couche supérieure
if couche < nb_couches - 1:
    goUp(pas)
    rotateRight(180)
```

Cette condition vérifie empêche le drone d'effectuer un décalage inutile et ordonne au drone de passer à la couche supérieure

```
except Exception as err:
    print(err)
    display()
    traceback.print_exc()
```

Cette partie appartient à un bloc `try`. Si une erreur survient pendant l'exécution du programme, on empêche le programme de s'arrêter brutalement et on affiche des informations utiles pour comprendre le problème.

```
# ===== PROGRAMME PRINCIPAL =====
if __name__ == '__main__':

    print("=== Création de la salle ===")
    LARGE_SALLE = int(input("Donner la longueur de la salle (en cm) : "))
    LONG_SALLE = int(input("donner la longueur de la salle (en cm) : "))
    HAUTEUR_SALLE = int(input("donner la hauteur de la salle (en cm) : "))

    room_str = f"(0 0, {LARGE_SALLE} 0, {LARGE_SALLE} {LONG_SALLE}, 0 {LONG_SALLE}, 0 0)"
    createRoom(room_str, HAUTEUR_SALLE)

    print("\n=== Création de la cible ===")
    x1 = int(input("Entrer la valeur minimal de la position de la cible sur l'axe X : "))
    x2 = int(input("Entrer la valeur maximal de la position de la cible sur l'axe X : "))
    y1 = int(input("Entrer la valeur minimal de la position de la cible sur l'axe Y : "))
    y2 = int(input("Entrer la valeur maximal de la position de la cible sur l'axe Y : "))
    z1 = int(input("Entrer la valeur minimal de la hauteur de la cible : "))
    z2 = int(input("Entrer la valeur maximale de la hauteur de la cible : "))

    createTargetIn(x1, y1, z1, x2, y2, z2)
    createDrone(DRONE_VIRTUAL, VIEWER_TKMPL, progfunc=monprogramme)
```

Et en fin l'exécution du programme principale. On constate sur cette partie aussi la création de la salle qui va être fait par l'utilisateur, ainsi que le placement de la cible

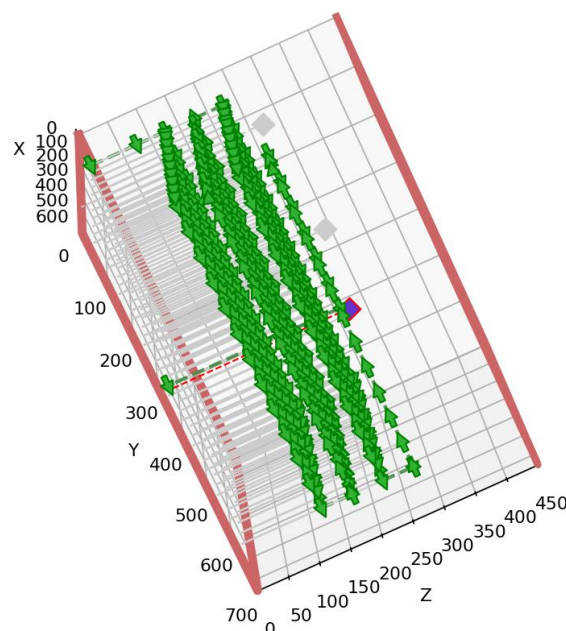
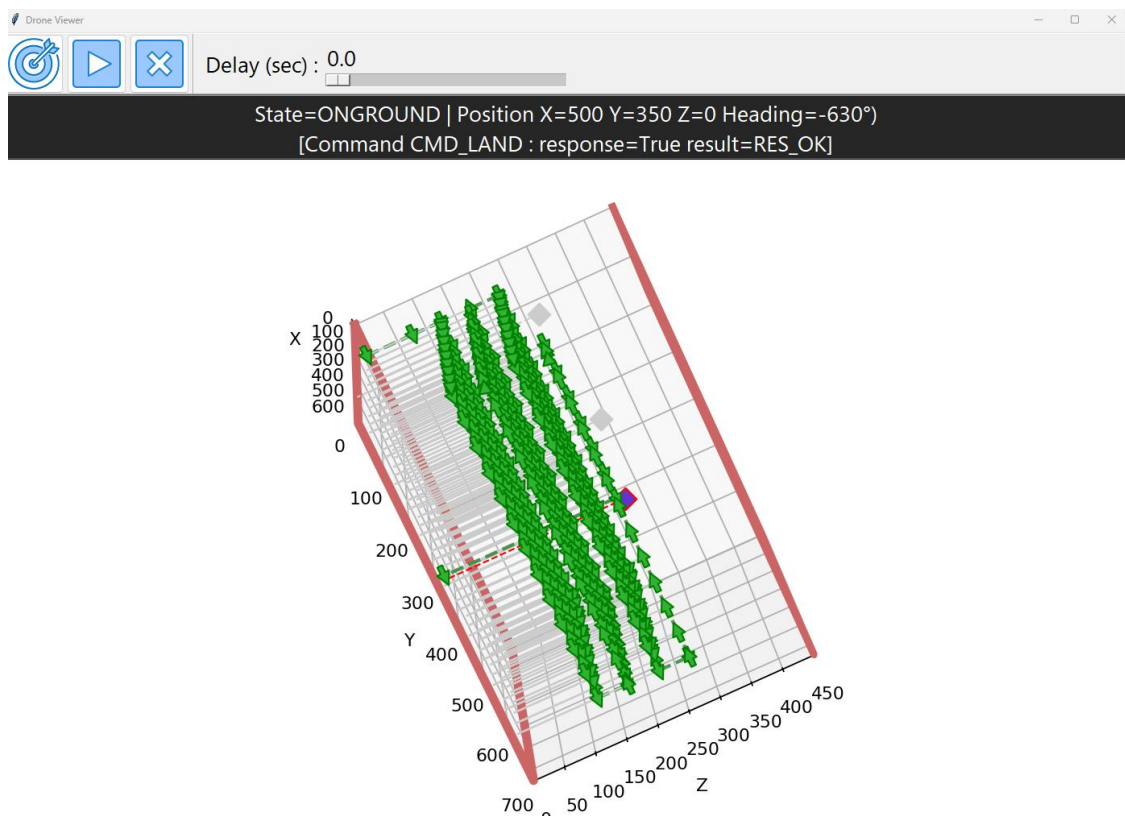
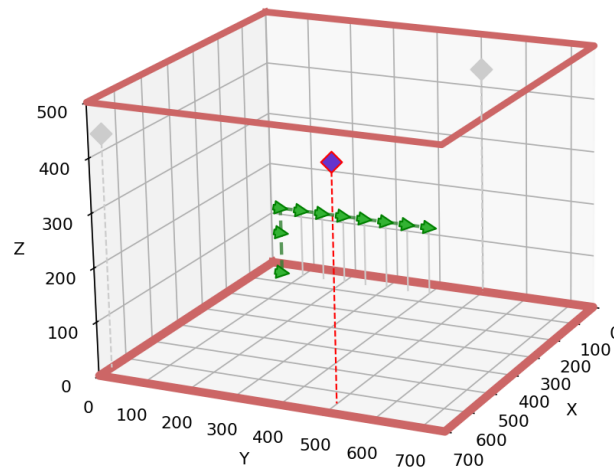
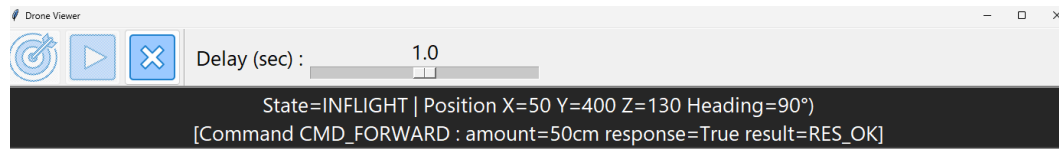
```
room_str = f"(0 0, {LARGE_SALLE} 0, {LARGE_SALLE} {LONG_SALLE}, 0 {LONG_SALLE}, 0 0)"
createRoom(room_str, HAUTEUR_SALLE)
```

Après la rentrée des dimensions par l'utilisateur, nous avons cette instruction qui permet de créer la salle en fonction des données fournies par l'utilisateur.

```
createTargetIn(x1, y1, z1, x2, y2, z2)
createDrone(DRONE_VIRTUAL, VIEWER_TKMPL, progfunc=monprogramme)
```

Nous avons cette instruction aussi qui permet de placer la cible en fonction des données fournies par l'utilisateur.

2. Exécution sur l'interface :



VI. Conclusion

Dans ce projet, nous avons conçu et implémenté un programme permettant à un drone virtuel de parcourir automatiquement une salle en trois dimensions afin de détecter une cible. L'algorithme repose sur un maillage de l'espace selon des pas réguliers, en tenant compte des contraintes de sécurité telles que la distance aux murs et la hauteur de vol. Le drone explore méthodiquement toutes les lignes, les colonnes et les différentes couches de hauteur de la salle jusqu'à ce que la cible soit détectée.

La mise en œuvre de structures de contrôle comme les boucles et les conditions a permis d'assurer une couverture complète de l'espace, tandis que l'utilisation d'un bloc try / except a facilité la gestion des erreurs et l'analyse des éventuels crashes du drone. Les tests réalisés ont mis en évidence l'importance d'un calcul précis des dimensions utiles et des déplacements, ainsi que d'un repositionnement correct du drone après chaque ligne et chaque couche.

Ce projet nous a permis de mieux comprendre les principes de la navigation autonome, de la programmation algorithmique et de la gestion des erreurs dans un environnement simulé. Il constitue une base solide pour des améliorations futures, telles que l'optimisation du parcours, l'ajout de capteurs plus intelligents ou l'adaptation dynamique du trajet en fonction des obstacles.

VII. Bibliographie

Il faut noter que pour réaliser ce projet, nous avons fait appel à l'ensemble des notions que nous avons vu au semestre 1. En plus de ça, nous avons bénéficié de l'appui d'un mentor.

Nous avons fait aussi l'usage de l'intelligence artificielle (IA) surtout au niveau des boucles **for**.

Pour terminer, ça a été un honneur pour nous de réaliser ce projet.